Document number:	P0899R0
Date:	2018-01-30
Project:	C++ Programming Language, Library Working Group
Reply-to:	Casey Carter <Casey@Carter.net>

LWG 3016 is Not a Defect

Abstract

LWG 3016 proposes adding the phrase:

> It is implementation-defined whether over-aligned types are supported (6.6.5 [basic.align]).

to the specification of `std::optional` in [optional.optional]/1. This proposal:

- provides rationale for why the change proposed by LWG 3016 is redundant,
- proposes closing LWG 3016 as NAD, and
- proposes removing similar redundant wording from the Standard Library specification.

Discussion

Absent explicit specification otherwise, it's reasonable to assume that the requirements of the core language apply to entities specified in the Standard Library. Specifically, the requirements in [basic.align] apply to the library portions of the standard. Hence, given the specification of a class template `S`:

```
template <class T>
struct S {
    T internal; // exposition only
};
```

There is no need to explicitly specify that [basic.align]/3 applies to "internal": if `T` is an over-aligned type, then support for `S<T>` is implementation-defined. Similarly, the specification of `optional<T>` in [optional.optional]/1:

> … When an instance of `optional<T>` *contains a value*, it means that an object of type `T`, referred to as the optional object's contained value, is allocated within the storage of the optional object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. The contained value shall be allocated in a region of the `optional<T>` storage suitably aligned for the type `T`. …

says that the contained value is "suitably aligned" for the type `T`. Albeit the term "suitably aligned" is not specified, I believe the only reasonable interpretation is that the contained value is aligned accordingly for a subobject of type `T` - just as in the first case of an exposition-only member - in an implementation-defined manner in accordance with [basic.align]/3.

Adding redundant text to [optional.optional]/1 would obviously not improve the quality of the Standard; if LWG agrees with the rationale provided herein, then LWG3016 should be closed as NAD.

Alternative

If LWG feels that the above interpretation is too subtle to be left implicit, we could add blanket wording in Clause 20 to cover the entire library rather than annotating each individual instance of a library component that

may create instances of a user-defined type with a cross-reference to [basic.align]. That requirement could take the form of a new paragraph at the end of Clause 20 (immediately after [lib.types.movedfrom]):

> 20.5.5.?? Alignment of program-defined types [lib.types.alignment]
>
> 1 [Note: Some library components allocate instances of program-defined object types, either within the storage of an instance of a library-defined object type, or dynamically. The extent to which over-alignment of such program-defined object instances is supported is implementation-defined ([basic.align]). –end note]

The author feels that such a note does not add significant value; it is presented here only to make LWG aware of the alternative.

Technical Specifications

In addition to closing LWG 3016 as NAD, it behooves us to remove similar redundant wording from the rest of the Standard Library specification. Wording relative to N4713.

Change [variant.variant]/1 as follows:

> 1 Any instance of `variant` at any given time either holds a value of one of its alternative types, or it holds no value. When an instance of `variant` holds a value of alternative type `T`, it means that a value of type `T`, referred to as the `variant` object's *contained value*, is allocated within the storage of the `variant` object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the contained value. The contained value shall be allocated in a region of the `variant` storage suitably aligned for all types in `Types...`. ~~It is implementation-defined whether over-aligned types are supported.~~

Change [meta.trans.other] as follows (Note that striking this sentence also resolves Editorial issue #1757 which is currently assigned LWG status):

> […]
>
> 2 ~~It is implementation-defined whether any extended alignment is supported (~~[~~basic.align~~]~~).~~
>
> […]

Change [depr.temporary.buffer] as follows:

> […]
>
> 2 *Effects:* Obtains a pointer to uninitialized, contiguous storage for `N` adjacent objects of type `T`, for some non-negative number `N`. ~~It is implementation-defined whether over-aligned types are supported (6.6.5).~~

Change [depr.default.allocator] as follows (Note: this makes the specification of the deprecated `allocator::allocate(size_t, const void*)` overload consistent with the non-deprecated `allocator::allocate(size_t)` overload):

> […]
>
> 3 *Returns:* A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. ~~It is implementation-defined whether over-aligned types are supported (~~[~~basic.align~~]~~).~~

4 *Remarks:* The storage is obtained by calling `::operator new~~(std::size_t)~~` ([new.delete]), but it is unspecified when or how often this function is called.